DATA MINING

Lecture Notes · Chapter 5

Mining Frequent Patterns, Associations, and Correlations

Basic Concepts and Methods

Prepared by Asif Zaman
Founder, Corpusian

Primary text J. Han, M. Kamber & J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed., Morgan Kaufmann, 2011 — Ch. 6 (pp. 243–278) and §§7.2–7.3

Topics Support & confidence · Apriori · Efficiency techniques · FP-growth · Interestingness & null-invariance · Closed/maximal, multilevel, multidimensional and constraint-based mining

5.1 Overview and Learning Objectives

This chapter is concerned with discovering recurring relationships — *patterns*, *associations* and *correlations* — that lie hidden in large transactional or relational datasets. The motivating application is **market basket analysis**: the often-quoted observation that "customers who buy diapers also tend to buy beer." Finding such associations automatically, across millions of transactions and without knowing in advance which items to examine, is the core problem the chapter addresses. The same machinery underpins recommendation, cross-selling, web-usage mining, intrusion detection and the discovery of co-occurring clinical findings.

Learning Objectives

On completing this chapter, students should be able to:

- **LO1** define frequent itemsets, support, confidence and lift, and state the Apriori property;
- **LO2** trace the Apriori algorithm step by step, including candidate generation and pruning;
- **LO3** explain and apply the FP-growth algorithm, which avoids candidate generation;
- **LO4** distinguish association rules from correlation, and compute lift, χ^2 and all-confidence;
- **LO5** describe pattern-evaluation measures and identify those that are robust to null-transaction inflation;
- **LO6** explain closed and maximal frequent itemsets and why they are preferred as compact representations;
- **LO7** apply multilevel and multidimensional association mining;
- **LO8** identify constraints that can be pushed into the mining process.

5.1.1 Transactions and Itemsets

A **transaction database** is a collection $D = \{T_1, T_2, \ldots, T_n\}$ in which each transaction T_i is a set of items drawn from a universe $I = \{i_1, i_2, \ldots, i_m\}$ and carries a unique transaction identifier (TID).

Table 1: Basic terminology.

Term	Formal definition	Example
Item	A single product or attribute value	Bread, Milk, Diaper
Itemset	A non-empty subset of items $X \subseteq I$	$\{\text{Bread}, \text{Milk}\}$
k-itemset	An itemset containing exactly k items	$\{\text{Beer}, \text{Diaper}, \text{Milk}\}$ is a 3-itemset
Transaction	A set of items bought together; has a unique TID	$T_1 = \{\text{Bread}, \text{Milk}, \text{Diaper}\}$
Database	D, the set of all transactions	$\lvert D\rvert = 10{,}000$ supermarket baskets

5.1.2 Support and Confidence

Two measures are central to association rules. The **support** of an itemset X is the fraction of transactions in D that contain it:

$$\text{support}(X) = \frac{|\{T \in D : X \subseteq T\}|}{|D|}, \qquad \text{support}(X \Rightarrow Y) = \text{support}(X \cup Y) = P(X \cup Y).$$

Note that $P(X \cup Y)$ denotes the probability that a transaction contains *both* X and Y (the union of the itemsets, not of the events).

Confidence measures the reliability of a rule: given that X occurs in a transaction, how often does Y occur as well?

$$\text{confidence}(X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)} = P(Y \mid X).$$

Strong Association Rule

A rule $X \Rightarrow Y$ (with $X, Y \neq \varnothing$ and $X \cap Y = \varnothing$) is called **strong** if

- $\text{support}(X \cup Y) \geq min_sup$ — the itemset is frequent, and
- $\text{confidence}(X \Rightarrow Y) \geq min_conf$ — the rule is reliable.

Both thresholds are set by the user.

5.1.3 Frequent Itemsets

An itemset X is a **frequent itemset** (historically, a *large itemset*) if $\text{support}(X) \geq min_sup$. Support may be quoted in two equivalent forms:

- **Absolute support** (support count): the number of transactions containing X, e.g. 500 transactions;
- **Relative support**: the fraction of transactions containing X, e.g. 5%.

Worked Example 5.1 — Support, Confidence and a Strong Rule

Transaction database D with $min_sup = 2$ transactions (40%) and $min_conf = 70\%$:

TID	Items purchased
T100	Bread, Milk
T200	Bread, Diaper, Beer, Eggs
T300	Milk, Diaper, Beer, Cola
T400	Bread, Milk, Diaper, Beer
T500	Bread, Milk, Diaper, Cola

Support counts: Bread $= 4$, Milk $= 4$, Diaper $= 4$, Beer $= 3$, Cola $= 2$, Eggs $= 1$.
Frequent 1-itemsets: $\{\text{Bread}\}$, $\{\text{Milk}\}$, $\{\text{Diaper}\}$, $\{\text{Beer}\}$, $\{\text{Cola}\}$ (Eggs is infrequent).
Frequent 2-itemsets: $\{\text{Bread, Milk}\}$:3, $\{\text{Bread, Diaper}\}$:3, $\{\text{Milk, Diaper}\}$:3, $\{\text{Diaper, Beer}\}$:3, $\{\text{Bread, Beer}\}$:2, $\{\text{Milk, Beer}\}$:2, $\{\text{Milk, Cola}\}$:2, $\{\text{Diaper, Cola}\}$:2.
Frequent 3-itemsets: $\{\text{Bread, Diaper, Beer}\}$:2, $\{\text{Milk, Diaper, Beer}\}$:2, $\{\text{Bread, Milk, Diaper}\}$:2, $\{\text{Milk, Diaper, Cola}\}$:2. No 4-itemset reaches the threshold.
Rule $\{\text{Diaper}\} \Rightarrow \{\text{Beer}\}$**:**

$$\text{support} = \tfrac{3}{5} = 60\%, \qquad \text{confidence} = \frac{\text{support}(\{\text{Diaper, Beer}\})}{\text{support}(\{\text{Diaper}\})} = \frac{3/5}{4/5} = 75\%.$$

Since $60\% \geq 40\%$ and $75\% \geq 70\%$, this is a **strong association rule**.

5.1.4 The Two-Step Framework

Association rule mining decomposes naturally into two steps:

Step 1. **Find all frequent itemsets** — enumerate every itemset X with $\text{support}(X) \geq min_sup$. This is the computationally expensive step.

Step 2. **Generate strong rules** — for each frequent itemset X and each non-empty proper subset $s \subset X$, output $(X - s) \Rightarrow s$ whenever its confidence is at least min_conf.

Key Insight

Once the frequent itemsets and their support counts are known, Step 2 needs no further database access: every confidence is a ratio of two support counts that are already available (by the Apriori property, every subset of a frequent itemset is itself frequent, so its count has been recorded). All the algorithmic effort therefore goes into Step 1.

5.2 Frequent Itemset Mining Methods

Three families of algorithms dominate, each with its own trade-offs.

Table 2: Families of frequent-itemset mining algorithms.

Family	Key idea	Representative	Main advantage
Apriori-based	Candidate generate-and-test; Apriori property prunes the search	Apriori (Agrawal & Srikant, 1994)	Simple and widely understood
Pattern-growth	Compress D into a tree; mine by recursive projection	FP-growth (Han, Pei & Yin, 2000)	No candidate generation; often an order of magnitude faster
Vertical format	Represent data as item $\rightarrow$ TID-set; intersect TID-sets	ECLAT (Zaki, 2000)	Fast support counting by intersection; suits dense data

5.2.1 The Apriori Property (Anti-Monotonicity)

Apriori Property

Every non-empty subset of a frequent itemset is also frequent. Equivalently, if an itemset X is infrequent ($\text{support}(X) < min_sup$), then every superset of X is also infrequent and can be pruned without being counted. This is the cornerstone of all level-wise frequent pattern mining algorithms.

Formal statement. For all itemsets X, Y:

$$X \subseteq Y \implies \text{support}(Y) \leq \text{support}(X).$$

Proof. If a transaction T contains Y, then $X \subseteq Y \subseteq T$, so T also contains X. Every transaction counted for Y is therefore counted for X, and $\text{support}(Y)$ can never exceed $\text{support}(X)$. □

5.3 The Apriori Algorithm

Proposed by Agrawal and Srikant (1994), Apriori performs a **level-wise, breadth-first** search. Each level alternates between two phases: generating candidate k-itemsets C_k from the frequent $(k-1)$-itemsets L_{k-1}, and scanning the database to count them.

5.3.1 Algorithm Outline

Algorithm 1 Apriori$(D,\ min_sup)$

Input: transaction database D; minimum support threshold min_sup
Output: L, the set of all frequent itemsets

$C_1 \leftarrow$ all single items occurring in D
$L_1 \leftarrow \{X \in C_1 : \text{support}(X) \geq min_sup\}$
$k \leftarrow 2$
while $L_{k-1} \neq \varnothing$ **do**
  $C_k \leftarrow \text{AprioriGen}(L_{k-1})$ ▷ *join + prune*
  for all transactions $t \in D$ **do** ▷ *one database scan*
    increment the count of every $c \in C_k$ with $c \subseteq t$
  $L_k \leftarrow \{X \in C_k : \text{support}(X) \geq min_sup\}$
  $k \leftarrow k + 1$
return $L = \bigcup_k L_k$

5.3.2 Candidate Generation: AprioriGen

Candidate generation has two sub-steps. Items within every itemset are kept in a fixed (e.g. lexicographic) order.

Join step

Candidate k-itemsets are produced by joining pairs of $(k-1)$-itemsets in L_{k-1} that share their first $k-2$ items:

```
INSERT INTO C_k
SELECT p.item_1, p.item_2, ..., p.item_{k-1}, q.item_{k-1}
FROM   L_{k-1} p, L_{k-1} q
WHERE  p.item_1 = q.item_1, ..., p.item_{k-2} = q.item_{k-2},
       p.item_{k-1} < q.item_{k-1}            - lexicographic order avoids duplicates
```

Prune step

A candidate is removed if any of its $(k-1)$-subsets is absent from L_{k-1}, because by the Apriori property it cannot be frequent:

Algorithm 2 Prune step of AprioriGen

```
for all itemsets c ∈ C_k do
    for all (k − 1)-subsets s of c do
        if s ∉ L_{k−1} then
            delete c from C_k                ▷ Apriori pruning
```

5.3.3 Detailed Worked Example

Worked Example 5.2 — Apriori, level by level

Database of five transactions, $min_sup = 2$ (40%).

TID	Itemset
T1	$\{A, C, T, W\}$
T2	$\{C, D, W\}$
T3	$\{A, C, T, W\}$
T4	$\{A, C, D, W\}$
T5	$\{A, C, D, T, W\}$

Pass 1 — frequent 1-itemsets

Itemset	Support	Frequent?
$\{A\}$	4	**Yes**
$\{C\}$	5	**Yes**
$\{D\}$	3	**Yes**
$\{T\}$	3	**Yes**
$\{W\}$	5	**Yes**

All five items are frequent, so $L_1 = \{A, C, D, T, W\}$.

Pass 2 — candidate 2-itemsets and L_2

Joining L_1 with itself yields all $\binom{5}{2} = 10$ pairs. Every 1-subset of each pair lies in L_1, so nothing is pruned from C_2. Scanning D gives:

Itemset	Count	Frequent?	Itemset	Count	Frequent?
$\{A, C\}$	4	**Yes**	$\{C, T\}$	3	**Yes**
$\{A, D\}$	2	**Yes**	$\{C, W\}$	5	**Yes**
$\{A, T\}$	3	**Yes**	$\{D, T\}$	1	**No**
$\{A, W\}$	4	**Yes**	$\{D, W\}$	3	**Yes**
$\{C, D\}$	3	**Yes**	$\{T, W\}$	3	**Yes**

Hence L_2 contains nine itemsets; only $\{D, T\}$ (count 1, occurring in T5 alone) is discarded.

Pass 3 — candidate 3-itemsets

The join step pairs members of L_2 sharing a first item. Prefix A ($\{A,C\}, \{A,D\}, \{A,T\}, \{A,W\}$) yields six candidates; prefix C ($\{C,D\}, \{C,T\}, \{C,W\}$) yields three; prefixes D and T each have only one partner in L_2 and yield none. For example, $\{A,C,D\}$ comes from $\{A,C\} + \{A,D\}$, and its 2-subsets $\{A,C\}$✓, $\{A,D\}$✓, $\{C,D\}$✓ are all in L_2, so it survives pruning.

Candidate	All 2-subsets in L_2?	Support	Frequent?
$\{A,C,D\}$	Yes	2	**Yes**
$\{A,C,T\}$	Yes	3	**Yes**
$\{A,C,W\}$	Yes	4	**Yes**
$\{A,D,T\}$	No — $\{D,T\} \notin L_2$	—	**Pruned**
$\{A,D,W\}$	Yes	2	**Yes**
$\{A,T,W\}$	Yes	3	**Yes**
$\{C,D,T\}$	No — $\{D,T\} \notin L_2$	—	**Pruned**
$\{C,D,W\}$	Yes	3	**Yes**
$\{C,T,W\}$	Yes	3	**Yes**

Two of the nine joined candidates are pruned *without* being counted. (An itemset such as $\{D,T,W\}$ is never even generated: the join would need $\{D,T\}$ in L_2.) Thus $|L_3| = 7$.

Pass 4 — candidate 4-itemsets

Only the prefix $\{A,C\}$ is shared by three members of L_3 ($\{A,C,D\}, \{A,C,T\}, \{A,C,W\}$), giving three joined candidates:

Candidate	All 3-subsets in L_3?	Support	Frequent?
$\{A,C,D,T\}$	No — $\{A,D,T\}, \{C,D,T\} \notin L_3$	—	**Pruned**
$\{A,C,D,W\}$	Yes	2 (T4, T5)	**Yes**
$\{A,C,T,W\}$	Yes	3 (T1, T3, T5)	**Yes**

Pass 5 — termination

The only join in L_4 is $\{A,C,D,W\} + \{A,C,T,W\}$, which yields $\{A,C,D,T,W\}$. This candidate is pruned because its subset $\{A,D,T,W\}$ is not in L_4. Hence $C_5 = \varnothing$ and the algorithm stops.

> **Summary of Example 5.2**
> $|L_1| = 5$, $|L_2| = 9$, $|L_3| = 7$, $|L_4| = 2$: a total of **23 frequent itemsets**, found with four database scans. The two maximal frequent itemsets are $\{A,C,D,W\}$ and $\{A,C,T,W\}$.

5.3.4 Generating Rules from Frequent Itemsets

Each frequent itemset X is partitioned into antecedent and consequent in every possible way:

Algorithm 3 Rule generation

```
for all frequent itemsets X with |X| ≥ 2 do
    for all non-empty proper subsets s ⊂ X do
        r ← (X − s) ⇒ s;    confidence(r) = support(X) / support(X − s)
        if confidence(r) ≥ min_conf then
            output r
```

Worked Example 5.3 — Rules from $X = \{A, C, W\}$

$\text{support}(X) = 4/5 = 80\%$; take $min_conf = 70\%$ as before. The relevant subset supports are $\{A, C\}$:4, $\{A, W\}$:4, $\{C, W\}$:5, $\{A\}$:4, $\{C\}$:5, $\{W\}$:5.

Rule	Confidence	Strong?
$\{A, C\} \Rightarrow \{W\}$	$\text{support}(\{A, C, W\}) / \text{support}(\{A, C\}) = 4/4 = 100\%$	Yes
$\{A, W\} \Rightarrow \{C\}$	$4/4 = 100\%$	Yes
$\{C, W\} \Rightarrow \{A\}$	$4/5 = 80\%$	Yes
$\{A\} \Rightarrow \{C, W\}$	$4/4 = 100\%$	Yes
$\{C\} \Rightarrow \{A, W\}$	$4/5 = 80\%$	Yes
$\{W\} \Rightarrow \{A, C\}$	$4/5 = 80\%$	Yes

5.3.5 Bottlenecks of Apriori

Table 3: Why Apriori struggles on large or dense data.

Problem	Cause	Impact
Multiple database scans	One full scan per itemset length k	I/O-intensive; slow for long patterns
Huge candidate sets	Combinatorial growth, e.g. $\binom{\lvert L_1 \rvert}{2}$ candidates in C_2; 10^4 frequent items give $\approx 5 \times 10^7$	Memory pressure and CPU cost
Counting overhead	Every transaction is matched against every candidate in C_k	Expensive when C_k is large

Moreover, a single frequent pattern of length 100 forces Apriori to generate on the order of $2^{100} \approx 10^{30}$ candidates, which is clearly infeasible.

5.4 Improving the Efficiency of Apriori

5.4.1 Hash-Based Itemset Counting

Idea (Park, Chen & Yu, 1995)**.** While scanning the database to count 1-itemsets, hash every 2-itemset of each transaction into a bucket of a hash table and increment the bucket count. A 2-itemset can be frequent only if its bucket is.

- A bucket is *frequent* if its count $\geq min_sup$ (a necessary, not sufficient, condition).

- Any 2-itemset hashing to an infrequent bucket is removed from C_2 before the second scan.
- This substantially reduces $|C_2|$, the usual bottleneck, at no extra scan.

5.4.2 Transaction Reduction

Idea. A transaction that contains no frequent k-itemset cannot contain any frequent $(k+1)$-itemset, so it can be marked and skipped in later scans.

- The effective database shrinks progressively from pass to pass.
- The saving is greatest in the later passes, when few long frequent itemsets remain.

5.4.3 Partitioning

Idea (Savasere, Omiecinski & Navathe, 1995)**.** Divide D into n non-overlapping partitions $P_1, \ldots, P_n$, each small enough to fit in memory. An itemset that is frequent in D must be locally frequent in at least one partition.

Algorithm 4 Partition-based mining (two scans)

Phase I ▷ *scan 1*

for all partitions P_i **do**
    mine local frequent itemsets with relative threshold min_sup, i.e. count $\geq min_sup \times |P_i|$
global candidates $\leftarrow$ union of all local frequent itemsets

Phase II ▷ *scan 2*

scan D once to count every global candidate; keep those with support $\geq min_sup$

Guarantee. If X fell below the relative threshold in every partition, its total count would fall below $min_sup \times |D|$; hence every globally frequent itemset appears among the candidates and there are *no false negatives*.

Advantage. Exactly two database scans, regardless of the maximum pattern length.

5.4.4 Sampling

Idea (Toivonen, 1996)**.** Mine a random sample $S \subset D$ in memory using a *lower* threshold $min_sup' < min_sup$, then verify the resulting itemsets against the full database.

- **Lower threshold:** makes it unlikely that a truly frequent itemset is missed in the sample.
- **Verification scan:** counts the sample-frequent itemsets (and their *negative border*) in D; if a border itemset turns out frequent, a second pass recovers the missed patterns.
- **Trade-off:** trades some accuracy risk for efficiency; an unrepresentative sample can miss globally frequent itemsets unless the extra pass is run.

5.4.5 Dynamic Itemset Counting

Idea (DIC; Brin *et al.*, 1997)**.** The database is divided into blocks marked by start points, and new candidates are added at any start point rather than only at the end of a full pass. As soon as all subsets of an itemset are known to be frequent, counting of that itemset begins.

- Fewer complete database scans than Apriori.

- More complex bookkeeping, since itemsets enter and leave the counting set mid-scan.

5.5 FP-growth: Mining Without Candidate Generation

Proposed by Han, Pei and Yin (2000), FP-growth avoids candidate generation entirely. It compresses the database into a **frequent pattern tree (FP-tree)** and then mines the tree recursively through *conditional pattern bases*.

> **FP-growth Key Insight**
> All of the frequency information needed to mine frequent patterns is retained in the FP-tree, a compact prefix-sharing structure. Mining proceeds by recursive divide-and-conquer on the compressed tree, and no candidate itemsets are ever generated.

5.5.1 FP-tree Construction

Step 1 — first database scan

Count the support of every item, discard infrequent items, and sort the remaining items in **descending order of support**. Call this list L; the same order is used throughout (ties are broken by any fixed rule).

Step 2 — second database scan: insert transactions

Create a root labelled `null`. Then, for each transaction:

1. delete its infrequent items and sort the rest in the order of L;
2. insert the sorted list as a path from the root, sharing any existing prefix;
3. increment the count of every shared node by 1, and create new nodes (count 1) for the unshared suffix;
4. link each new node into the **header table** chain for its item, via node-links.

Table 4: Components of an FP-tree.

Component	Purpose
Root node (`null`)	Starting point of every path; carries no item label
Node	Stores item name, count, parent pointer and node-link
Node-link	Chains all nodes carrying the same item; used during mining
Header table	One entry per frequent item: its global support and a pointer to the first node in its node-link chain

5.5.2 FP-tree Construction — Worked Example

Worked Example 5.4 — Building the FP-tree ($min_sup = 3$)

TID	Original transaction	Frequent items, ordered by L
T1	$\{f, a, c, d, g, i, m, p\}$	$\langle f, c, a, m, p\rangle$
T2	$\{a, b, c, f, l, m, o\}$	$\langle f, c, a, b, m\rangle$
T3	$\{b, f, h, j, o\}$	$\langle f, b\rangle$
T4	$\{b, c, k, s, p\}$	$\langle c, b, p\rangle$
T5	$\{a, f, c, e, l, p, m, n\}$	$\langle f, c, a, m, p\rangle$

Frequent items (support ≥ 3): $L = \langle f{:}4,\ c{:}4,\ a{:}3,\ b{:}3,\ m{:}3,\ p{:}3\rangle$. Items l and o (support 2) and all others (support 1) are removed.

Insertion trace

- T1 $\langle f, c, a, m, p\rangle$: new path $\texttt{null} \to f{:}1 \to c{:}1 \to a{:}1 \to m{:}1 \to p{:}1$.
- T2 $\langle f, c, a, b, m\rangle$: share $f{:}2 \to c{:}2 \to a{:}2$; new branch $a \to b{:}1 \to m{:}1$.
- T3 $\langle f, b\rangle$: share $f{:}3$; new branch $f \to b{:}1$.
- T4 $\langle c, b, p\rangle$: no common prefix (does not start with f); new branch $\texttt{null} \to c{:}1 \to b{:}1 \to p{:}1$.
- T5 $\langle f, c, a, m, p\rangle$: identical to T1, so the entire path is shared: $f{:}4 \to c{:}3 \to a{:}3 \to m{:}2 \to p{:}2$. No new nodes.

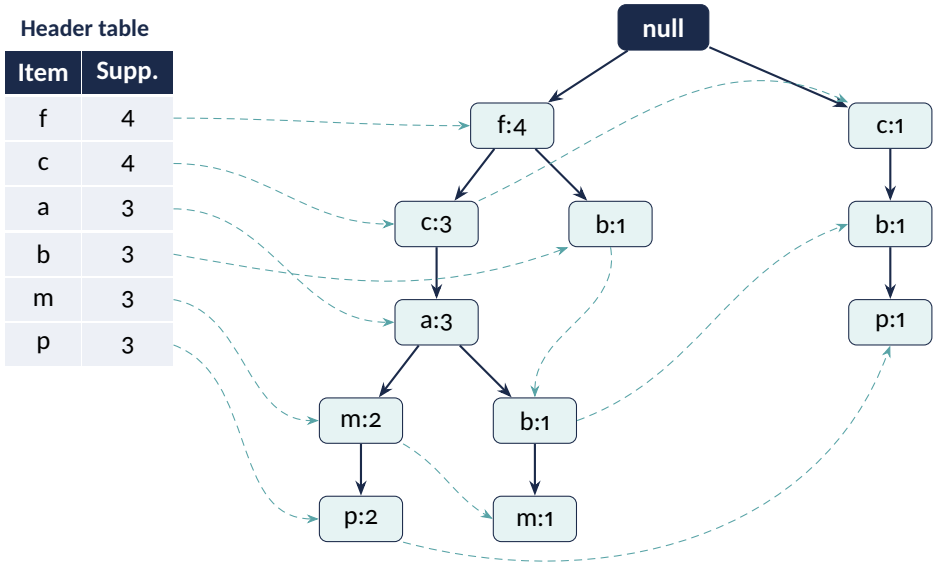


Figure 1: The FP-tree of Worked Example 5.4. Solid arrows are tree edges; dashed teal arrows are node-links threading all nodes of the same item from the header table. Five transactions are stored in 11 nodes.

5.5.3 The FP-growth Mining Algorithm

Algorithm 5 FP-growth($Tree,\ \alpha$) — initial call: FP-growth(FP-tree, $\varnothing$)

```
if Tree consists of a single path P then
    for all combinations β of the nodes in P do
        output β ∪ α with support = minimum count of the nodes in β
else
    for all items a_i in the header table of Tree do          ▷ least frequent first
        β ← a_i ∪ α, with support(β) = support(a_i); output β
        CPB ← prefix paths of all a_i-nodes, counted as their a_i-node   ▷ pattern base
        Tree_β ← FP-tree built from CPB with items of count ≥ min_sup    ▷ cond. FP-tree
        if Tree_β ≠ ∅ then
            FP-growth(Tree_β, β)                              ▷ recurse
```

5.5.4 Conditional Pattern Bases and Conditional FP-trees

Mining the tree of Figure 1 bottom-up through the header table gives:

Table 5: Mining the FP-tree of Example 5.4 ($min_sup = 3$).

Item	Conditional pattern base	Conditional FP-tree	Frequent patterns generated
p	$\{f, c, a, m{:}2\}, \{c, b{:}1\}$	$\langle c{:}3\rangle$ (f, a, m reach only 2; b only 1)	$\{c, p\}{:}3$
m	$\{f, c, a{:}2\}, \{f, c, a, b{:}1\}$	$\langle f{:}3, c{:}3, a{:}3\rangle$	$\{f, m\}, \{c, m\}, \{a, m\}$, $\{f, c, m\}, \{f, a, m\}, \{c, a, m\}$, $\{f, c, a, m\}$, all $:3$
b	$\{f, c, a{:}1\}, \{f{:}1\}, \{c{:}1\}$	$\varnothing$ ($f{:}2, c{:}2, a{:}1$)	none beyond $\{b\}{:}3$
a	$\{f, c{:}3\}$	$\langle f{:}3, c{:}3\rangle$	$\{f, a\}, \{c, a\}, \{f, c, a\}$, all $:3$
c	$\{f{:}3\}$ (the root-level $c{:}1$ has an empty prefix)	$\langle f{:}3\rangle$	$\{f, c\}{:}3$
f	$\varnothing$	$\varnothing$	none beyond $\{f\}{:}4$

Together with the six frequent single items, FP-growth returns $6 + 1 + 7 + 0 + 3 + 1 + 0 = \mathbf{18}$ frequent itemsets — exactly the result that Apriori would produce on the same data, obtained here without generating a single candidate.

5.5.5 FP-growth versus Apriori

Table 6: Apriori and FP-growth compared.

Dimension	Apriori	FP-growth
Database scans	Up to k (or $k+1$) scans for maximum pattern length k	Exactly 2 (one to count items, one to build the tree)
Candidate generation	Explicit: join + prune	None; patterns grow directly from the tree
Memory	Candidate sets C_k can be huge	FP-tree is compact (shared prefixes), but must fit in memory or be partitioned
Speed	Slow for long patterns or low min_sup	Often an order of magnitude faster; scales better
Implementation	Simple	More involved (tree, node-links, projection)
Best suited to	High min_sup, short patterns	Low min_sup, long patterns, dense data

5.6 Pattern Evaluation: Interestingness Measures

Support and confidence alone can mislead. A rule may have high confidence and still be uninteresting — or actively misleading — when the consequent is already very common. Additional measures of **interestingness**, above all **correlation**, are needed.

Worked Example 5.5 — A strong but misleading rule

Of $10{,}000$ transactions, $6{,}000$ include computer games, $7{,}500$ include videos, and $4{,}000$ include both. With $min_sup = 30\%$ and $min_conf = 60\%$, the rule

$$\text{buys(game)} \Rightarrow \text{buys(video)} \qquad [\text{support} = 40\%,\ \text{confidence} = 66.7\%]$$

is strong. Yet $P(\text{video}) = 75\% > 66.7\%$: buying a game actually *lowers* the probability of buying a video. The two purchases are negatively correlated, as the measures below confirm.

5.6.1 Lift

$$\text{lift}(X \Rightarrow Y) = \frac{\text{confidence}(X \Rightarrow Y)}{\text{support}(Y)} = \frac{P(X \cup Y)}{P(X)\,P(Y)}.$$

Lift value	Interpretation
$\text{lift} > 1$	X and Y are positively correlated (co-occur more often than chance predicts)
$\text{lift} = 1$	X and Y are independent
$\text{lift} < 1$	X and Y are negatively correlated (the presence of X suppresses Y)

Lift — two cases

Example 5.1, $\{$**Diaper**$\} \Rightarrow \{$**Beer**$\}$**:** $\text{support}(\text{Beer}) = 0.60$, $\text{confidence} = 0.75$, so $\text{lift} = 0.75/0.60 = 1.25$. Beer is 25% more likely when diapers are bought: a positive correlation.
Example 5.5, game ⇒ video: $\text{lift} = 0.40/(0.60 \times 0.75) = 0.89 < 1$: a negative correlation, despite the high confidence.

5.6.2 The Chi-Squared (χ^2) Correlation Test

$$\chi^2 = \sum_{\text{cells}} \frac{(\text{Observed} - \text{Expected})^2}{\text{Expected}}, \qquad E_{ij} = \frac{\text{row total}_i \times \text{column total}_j}{|D|}.$$

The test is applied to the 2×2 contingency table of item X (present/absent) against item Y (present/absent); $n(\cdot)$ denotes a transaction count:

	Y	$\neg Y$	**Row total**
X	$n(X \wedge Y)$	$n(X \wedge \neg Y)$	$n(X)$
$\neg X$	$n(\neg X \wedge Y)$	$n(\neg X \wedge \neg Y)$	$\lvert D\rvert - n(X)$
Column total	$n(Y)$	$\lvert D\rvert - n(Y)$	$\lvert D\rvert$

A 2×2 table has **one degree of freedom**; the critical value at the 0.05 significance level is $\chi^2_{0.05,1} = 3.841$.

- $\chi^2 > 3.841$: reject independence — X and Y are correlated.
- χ^2 tests *whether* a dependence exists, not how strong it is; its direction is read by comparing the observed and expected counts in the (X, Y) cell.

Worked Example 5.6 — χ^2 for game and video

Expected counts are shown in parentheses.

	video	¬video	Total
game	4,000 (4,500)	2,000 (1,500)	6,000
¬game	3,500 (3,000)	500 (1,000)	4,000
Total	7,500	2,500	10,000

$$\chi^2 = \frac{500^2}{4500} + \frac{500^2}{1500} + \frac{500^2}{3000} + \frac{500^2}{1000} = 55.6 + 166.7 + 83.3 + 250.0 = 555.6.$$

Since $555.6 \gg 3.841$, game and video are correlated; because the observed count in the (game, video) cell, 4,000, is *below* its expectation of 4,500, the correlation is negative — in agreement with $\text{lift} = 0.89$.

5.6.3 The Null-Transaction Problem

A **null transaction** with respect to X and Y is one that contains neither. In real baskets null transactions usually vastly outnumber the rest, and any measure whose value depends on their number becomes unreliable: changing nothing but the amount of unrelated data can make an association look stronger or weaker.

Worked Example 5.7 — Lift drifts with the number of null transactions

Let X = Caviar and Y = Truffles, each bought in 100 transactions, with 90 transactions containing both. Only $|D|$ — and hence the number of null transactions, $|D| - (100 + 100 - 90) = |D| - 110$ — is varied:

$\lvert D\rvert$	Null trans.	lift	confidence$(X \Rightarrow Y)$	Kulczynski	All-conf.
1,000	890	9	0.90	0.90	0.90
10,000	9,890	90	0.90	0.90	0.90
100,000	99,890	900	0.90	0.90	0.90

For $|D| = 10{,}000$: $\text{support}(X \cup Y) = 90/10{,}000 = 0.9\%$ (not frequent under any reasonable threshold), and $\text{lift} = 0.009/(0.01 \times 0.01) = 90$. The relationship between caviar and truffles is *identical* in all three rows, yet lift swings over two orders of magnitude, purely because unrelated transactions were added. Confidence, Kulczynski and all-confidence depend only on $n(X)$, $n(Y)$ and $n(X \wedge Y)$, and remain at 0.90: nine in ten caviar buyers also buy truffles, and vice versa.

5.6.4 Null-Invariant Measures

A measure is **null-invariant** if its value does not change when null transactions are added or removed.

Table 7: Interestingness measures and null-invariance.

Measure	Formula	Range	Null-invariant?
All-confidence	$\text{support}(X \cup Y) \,/\, \max\{\text{support}(X), \text{support}(Y)\}$ $= \min\{P(X\lvert Y), P(Y\lvert X)\}$	$[0, 1]$	**Yes**
Max-confidence	$\max\{P(X\lvert Y),\ P(Y\lvert X)\}$	$[0, 1]$	**Yes**
Kulczynski (Kulc)	$\frac{1}{2}\left[P(X\lvert Y) + P(Y\lvert X)\right]$	$[0, 1]$	**Yes**
Cosine	$P(X \cup Y) \,/\, \sqrt{P(X)\,P(Y)}$	$[0, 1]$	**Yes**
Confidence	$P(Y\lvert X) = P(X \cup Y) \,/\, P(X)$	$[0, 1]$	**Yes** (but asymmetric)
Lift	$P(X \cup Y) \,/\, \big(P(X)\,P(Y)\big)$	$[0, \infty)$	**No**
χ^2	$\sum (O - E)^2 / E$	$[0, \infty)$	**No**
Support (relative)	$P(X \cup Y)$	$[0, 1]$	**No**

Recommendation

Confidence is null-invariant, but it looks in one direction only and ignores how common Y is, which is exactly how Example 5.5 misled us. The symmetric null-invariant measures fix both problems. For rare-but-interesting patterns, or whenever null transactions dominate, prefer **Kulczynski** together with the **imbalance ratio**, or **all-confidence**. Reserve lift and χ^2 for settings where null-transaction inflation is not a concern, such as items of moderate support.

For Example 5.5: $\text{Kulc} = \frac{1}{2}(4000/6000 + 4000/7500) = 0.60$, all-confidence $= 4000/7500 = 0.53$ and cosine $= 0.60$ — all moderate, and unchanged however many unrelated transactions are added to the database.

5.6.5 Imbalance Ratio

Kulczynski alone cannot reveal whether its value arises from two balanced conditional probabilities or from one large and one small value. The **imbalance ratio** (IR) complements it:

$$\begin{aligned}\text{IR}(X, Y) &= \frac{|\,\text{support}(X) - \text{support}(Y)|}{\text{support}(X) + \text{support}(Y) - \text{support}(X \cup Y)} \\ &= \frac{|P(X|Y) - P(Y|X)|}{P(X|Y) + P(Y|X) - P(X|Y)\,P(Y|X)}.\end{aligned}$$

(The two forms are algebraically equivalent: multiply numerator and denominator of the second by $\text{support}(X)\,\text{support}(Y)/\,\text{support}(X \cup Y)$.)

- $\text{IR} = 0$: perfectly balanced — the association is equally strong in both directions.
- $\text{IR} \to 1$: highly imbalanced — one direction is much stronger than the other.

For Example 5.5, $\text{IR} = |6000 - 7500|/(6000 + 7500 - 4000) = 0.16$: fairly balanced.

5.7 Mining Various Kinds of Association Rules

5.7.1 Closed and Maximal Frequent Itemsets

The complete set of frequent itemsets can be exponentially large. Two compressed representations are widely used:

Table 8: Compact representations of the frequent itemsets.

Concept	Definition	Property
Closed frequent itemset	X is frequent and no proper superset $Y \supset X$ has the same support as X	**Lossless**: the exact support of every frequent itemset can be derived
Maximal frequent itemset	X is frequent and no proper superset of X is frequent	**Lossy**: tells which itemsets are frequent, but not their exact supports

Why they matter. A database containing a single frequent pattern of length 100 already has

$$\binom{100}{1} + \binom{100}{2} + \cdots + \binom{100}{100} = 2^{100} - 1 \approx 1.27 \times 10^{30}$$

frequent itemsets, since every non-empty subset is frequent. Closed itemsets compress this without any loss of information; maximal itemsets compress even further but lose the support values of their subsets.

> **Closed and maximal itemsets in Example 5.2**
>
> Of the 23 frequent itemsets, $\{A\}$:4 is *not* closed, because its superset $\{A, C\}$ also has support 4; likewise $\{A, W\}$ is not closed ($\{A, C, W\}$:4). By contrast $\{C, W\}$:5 is closed. The maximal fre-

quent itemsets are $\{A, C, D, W\}$:2 and $\{A, C, T, W\}$:3: from them alone we know that $\{A, D\}$ is frequent, but not that its support is 2.

Subset relationships

$$\text{Maximal frequent} \subseteq \text{Closed frequent} \subseteq \text{Frequent itemsets}.$$

Every maximal frequent itemset is closed (any superset with equal support would itself be frequent), and every closed frequent itemset is, by definition, frequent.

5.7.2 Multilevel Association Rules

Items are often organised into *concept hierarchies*, e.g. 2% milk → milk → dairy → food. Rules can be mined at different levels of granularity:

- **Higher level (generic):** "dairy products ⇒ baked goods" — high support, low specificity.
- **Lower level (specific):** "2% milk ⇒ white bread" — lower support, higher specificity.

Reduced minimum support

Items lower in the hierarchy naturally have lower support, so a single uniform threshold either misses the specific rules (if high) or floods the output with generic ones (if low). A common remedy is a **reduced minimum support at lower levels**:

Level	min_sup	Example items
Level 1	5%	dairy, baked goods
Level 2	2%	milk, bread
Level 3	0.5%	2% milk, whole-wheat bread

Redundancy filtering

Rule R_1 is an *ancestor* of rule R_2 if R_1 is obtained by replacing items of R_2 with their ancestors in the hierarchy. R_2 is **redundant** if its support and confidence are close to the values expected from R_1, so it adds no new information.

Redundant descendant rule

R_1: milk ⇒ wheat bread [support 8%, confidence 70%]
R_2: 2% milk ⇒ wheat bread [support 2%, confidence 72%]
If about one quarter of milk sold is 2% milk, the expected support of R_2 is $8\% \times \frac{1}{4} = 2\%$ and its expected confidence about 70%. R_2 matches these expectations, so it is redundant and can be pruned.

5.7.3 Multidimensional Association Rules

Single-dimensional rules involve one predicate, typically buys:

$$\text{buys}(X, \text{"digital camera"}) \Rightarrow \text{buys}(X, \text{"HP printer"}).$$

Multidimensional rules involve two or more dimensions (predicates):

- **Inter-dimensional rules** — no predicate is repeated:
  $\text{age}(X, \text{“30–39”}) \wedge \text{income}(X, \text{“40K–50K”}) \Rightarrow \text{buys}(X, \text{“laptop”})$.
- **Hybrid-dimensional rules** — some predicate (usually buys) occurs more than once:
  $\text{age}(X, \text{“20–29”}) \wedge \text{buys}(X, \text{“laptop”}) \Rightarrow \text{buys}(X, \text{“HP printer”})$.

Quantitative attributes

Numeric attributes such as age or income must be discretised before, or during, mining:

- **Static discretisation:** predefined ranges or a concept hierarchy (child / youth / adult / senior), fixed before mining.
- **Dynamic discretisation:** ranges chosen during mining so as to maximise support or confidence (quantitative association rules); more flexible but computationally costlier.
- **Clustering-based:** cluster the numeric values and mine rules over the clusters, respecting the distribution of the data.

5.7.4 Constraint-Based Association Mining

Not every frequent pattern is of interest to the analyst. Constraints inject user knowledge into the mining process, focusing discovery and — where they can be pushed deep into the algorithm — reducing its cost.

Table 9: Types of mining constraints.

Constraint type	Example	Pushed into mining?
Knowledge type	Mine association rules only (not sequences or classifiers)	Yes — choice of algorithm
Data	Consider only transactions from 2023	Yes — pre-filter D
Dimension / level	Mine at product-category level only	Yes — choice of hierarchy level
Interestingness	$\text{lift} \geq 1.5$, confidence $\geq 80\%$	Yes — threshold pruning
Rule	Total profit of items in the RHS $\geq \$500$	Partially — depends on the property of the constraint (below)

Anti-monotone constraints (pushable)

Definition. A constraint C is *anti-monotone* if, whenever an itemset X violates C, every superset of X also violates C.

- **Example:** $\text{sum}(\text{price}) \leq \$100$ (prices non-negative). Once a set of items costs more than \$100, adding items cannot reduce the cost.
- Such constraints are pushed into Apriori's pruning step exactly like the support threshold, which is itself anti-monotone.

Monotone constraints (also pushable)

Definition. A constraint C is *monotone* if, whenever an itemset X satisfies C, every superset of X also satisfies C.

- **Example:** $\text{sum}(\text{price}) \geq \$10$. Once a set costs at least \$10, adding items can only increase its cost.
- Once an itemset satisfies the constraint, none of its supersets needs to be checked against it again.

References

[1] J. Han, M. Kamber and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann, 2011, Ch. 6–7.

[2] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. VLDB*, 1994, pp. 487–499.

[3] J. Han, J. Pei and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. ACM SIGMOD*, 2000, pp. 1–12.

[4] J. S. Park, M.-S. Chen and P. S. Yu, "An effective hash-based algorithm for mining association rules," in *Proc. ACM SIGMOD*, 1995, pp. 175–186.

[5] A. Savasere, E. Omiecinski and S. Navathe, "An efficient algorithm for mining association rules in large databases," in *Proc. VLDB*, 1995, pp. 432–443.

[6] H. Toivonen, "Sampling large databases for association rules," in *Proc. VLDB*, 1996, pp. 134–145.

[7] S. Brin, R. Motwani, J. D. Ullman and S. Tsur, "Dynamic itemset counting and implication rules for market basket data," in *Proc. ACM SIGMOD*, 1997, pp. 255–264.

[8] M. J. Zaki, "Scalable algorithms for association mining," *IEEE Trans. Knowledge and Data Engineering*, 12(3):372–390, 2000.

[9] T. Wu, Y. Chen and J. Han, "Re-examination of interestingness measures in pattern mining: a unified framework," *Data Mining and Knowledge Discovery*, 21(3):371–397, 2010.